```
    res.set('Vary', 'Origin');
  } else {
    res.set('Access-Control-Allow-Origin', '*');
  }
```

Even though the code in the listing checks if the origin is valid, there is no `else` block to define how the request behaves with an invalid origin. Without defining this behavior, the code continues processing the request, even if the request is from an unauthorized origin. This means that even if the origin is invalid, the request's action, such as returning the user's blog posts, will go through.

What you really want to do is short-circuit the request in the CORS handler and return an error to the user. By stopping the request at the CORS-level, you prevent any other request code from running. The following listing introduces a new option to short-circuit the request.

Listing 6.7 Short-circuiting the request on invalid origins.

```
var originWhitelist = [
  'null',
  'http://localhost:1111'
];

var corsOptions = {
  allowOrigin: createWhitelistValidator(originWhitelist),
  allowCredentials: true,
  shortCircuit: true                                    <-- Introduces a new shortCircuit option.
};

var handleCors = function(options) {
  return function(req, res, next) {

    if (options.allowOrigin) {
      var origin = req.headers['origin'];
      if (options.allowOrigin(origin)) {
        res.set('Access-Control-Allow-Origin', origin);
      } else if (options.shortCircuit) {
        res.status(403).end();                          <-- If origin is invalid, stops processing request and returns an HTTP 403 response.
        return;
      }
      res.set('Vary', 'Origin');
    } else {
      res.set('Access-Control-Allow-Origin', '*');
    }
```

The code begins by introducing a new `shortCircuit` option that controls this new behavior. Next, the code returns an HTTP 403 status if the origin is invalid, and stops processing the request.

ADDITIONAL CSRF PROTECTION

Validating the origin is a nice way to verify that the request meets your expectations and for most use cases, this should be a sufficient form of CSRF protection. However, it